# Visualization of Network Data

Xu Shirong

School of Economics, Xiamen University

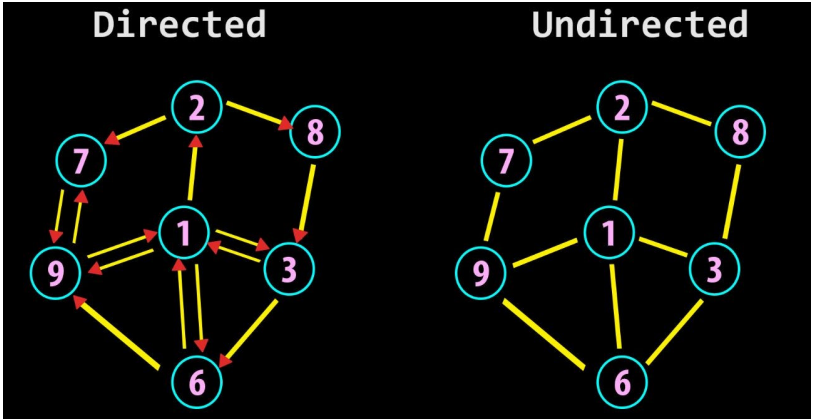
2026

# What is Network Data?

- Data represented as a **graph**
- **Nodes**: entities
- **Edges**: relationships
- Examples:
  - Social networks
  - Citation networks
  - Transportation

# Types of Network

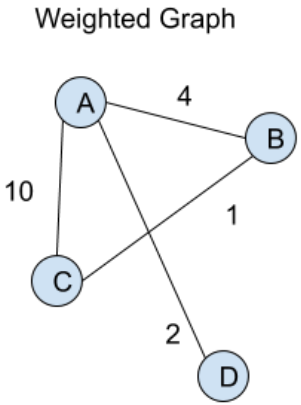
Undirected vs Directed

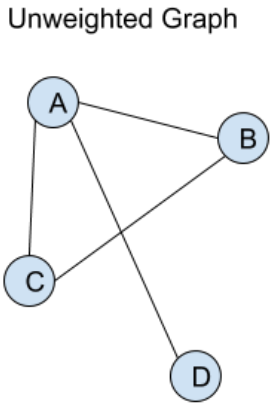



Can you give some examples for these two?

# Types of Network

Weighted and Unweighted graphs




Can you give some examples for these two?

# Data Representation

**Adjacency Matrix**

- $N \times N$
- Memory heavy
- Often sparse

**Incidence Matrix**

- Node-edge relation

**Edge List**

- Most common
- Flexible

**Example**

(A, B)
(A, C)
(B, C)

# Why Visualization Matters

- Understand **structure**
- Detect **clusters**
- Identify **important nodes**

**Problem**

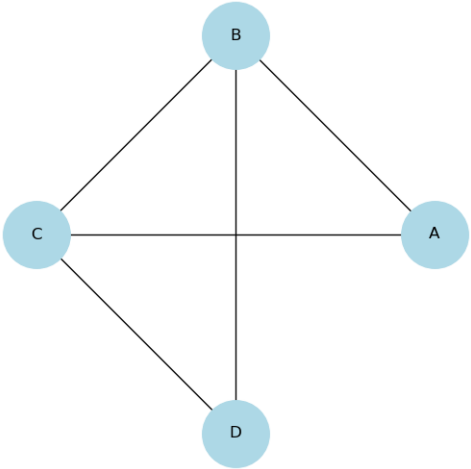
Dense networks $\rightarrow$ **Hairball Graph**

# Create Undirected Graph (NetworkX)

```
import networkx as nx
import matplotlib
matplotlib.use("TkAgg")
import matplotlib.pyplot as plt

G = nx.Graph()  # Undirected Graph
G.add_edge("A", "B")
G.add_edge("A", "C")
G.add_edge("B", "C")
G.add_edge("C", "D")
G.add_edge("B", "D")

pos = nx.circular_layout(G)
plt.figure(figsize=(5, 5))
nx.draw(G, pos,with_labels=True,
    node_color="lightblue",
    node_size=2500
)
plt.axis("off")
plt.show()
```

- Undirected graph
- Edges have no direction
- Useful for symmetric relationships
- `G.add_edge("A","B")`: means A is connected with B.

# Graph Plot




We can change the following things for better visualization

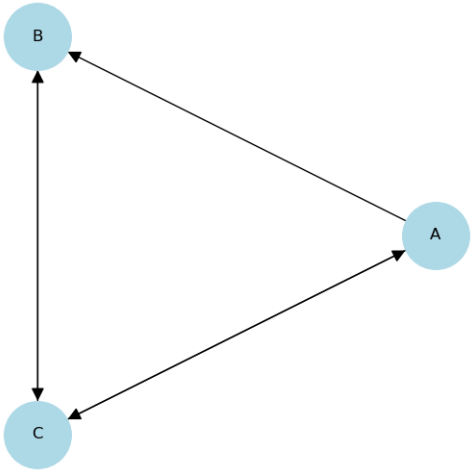
- Node Size and Color
- Line Size and Color
- Thickness of arrows
- Labels

# Create Directed Graph (NetworkX)

```
import networkx as nx
import matplotlib
import matplotlib.pyplot as plt
matplotlib.use("TkAgg")
G = nx.DiGraph() # Directed Graph
G.add_edge("A", "B")
G.add_edge("A", "C")
G.add_edge("B", "C")
G.add_edge("C", "A")
G.add_edge("C", "B")

pos = nx.circular_layout(G)
plt.figure(figsize=(5, 5))
nx.draw(G, pos, with_labels=True,
    node_color="lightblue", node_size
    =2500, arrowsize=20)
plt.title("Creating a graph")
plt.axis("off")
plt.show()
```

- Directed graph
- Easy to construct
- Supports attributes
- `G.add_edge("A","B")`: means include a directed edge from A pointing to B.

# Graph Plot




We can change the following things for better visualization

- Node Size and Color
- Arrow Size and Color
- Thickness of arrows
- Labels

# Show the information of Directed Network

```
print(G.nodes())  # List of nodes
print(G.edges())  # List of edges
print(G.degree())  # Node degrees
# ["A", "B", "C"]
# [("A", "B"), ("A", "C"), ("B", "C"), ("C", "A"), ("C", "B")]
# [("A", 3), ("B", 3), ("C", 4)]
```

- Can you guess the definition of degree of a node?
- What are practical definitions of degree?

# Show the information of Directed Network

```
print(G.nodes())  # List of nodes
print(G.edges())  # List of edges
print(G.degree())  # Node degrees
# ["A", "B", "C"]
# [("A", "B"), ("A", "C"), ("B", "C"), ("C", "A"), ("C", "B")]
# [("A", 3), ("B", 3), ("C", 4)]
```

- Can you guess the definition of degree of a node?
- What are practical definitions of degree?
- In Social Networks, degree represents activity level / social connectivity and high degree indicates higher exposure and potential influence.

# In and Out Degrees

```
in_degree = G.in_degree()
out_degree = G.out_degree()
print("In-degree:", in_degree)
print("Out-degree:", out_degree)
# In-degree: [("A", 1), ("B", 2), ("C", 2)]
# Out-degree: [("A", 2), ("B", 1), ("C", 2)]
```

- Can you guess the definition of in/out degrees of a node?
- What are practical definitions of in/out degrees?

# In and Out Degrees

```
in_degree = G.in_degree()
out_degree = G.out_degree()
print("In-degree:", in_degree)
print("Out-degree:", out_degree)
# In-degree: [("A", 1), ("B", 2), ("C", 2)]
# Out-degree: [("A", 2), ("B", 1), ("C", 2)]
```

- Can you guess the definition of in/out degrees of a node?
- What are practical definitions of in/out degrees?
- In Social networks, in and out degrees can represent the number of people you follow and you are followed.

# Mathematical Representations of Graph Data

**Definition:**

- A graph $G = (V, E)$
- $V$: set of nodes (vertices)
- $E$: set of edges

**Examples:**

- Social networks
- Transportation systems
- Communication networks

# Adjacency Matrix

**Idea:**

- Represent a graph using a matrix

**Definition:**

$$A_{ij} = \begin{cases} 1, & \text{if there is an edge between } i \text{ and } j \\ 0, & \text{otherwise} \end{cases}$$

**Matrix size:**

$$A \in \mathbb{R}^{n \times n}$$

where $n = |V|$

# Example Graph

Consider a graph with 4 nodes:

- Edges: $(1,2),(1,3),(2,4)$

**Adjacency matrix:**

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

**Observation:**

- Symmetric (undirected graph)

# Directed Graph

**Directed edge:** $i \to j$

**Adjacency matrix:**

$$A_{ij} = \begin{cases} 1, & \text{if edge } i \to j \\ 0, & \text{otherwise} \end{cases}$$

**Key difference:**

- Matrix is NOT symmetric

# Degree and Adjacency Matrix

**Degree of node $i$:**

$$d_i = \sum_{j=1}^{n} A_{ij}$$

**Matrix form:**

$$\mathbf{d} = A\mathbf{1}$$

**Interpretation:**

- Row sum = number of connections

# Important Properties

1. **Symmetry**
   - Undirected graph $\Rightarrow A = A^T$
2. **Sparsity**
   - Most real graphs are sparse
3. **Diagonal**
   - $A_{ii} = 0$ (no self-loop)

# Why Adjacency Matrix?

**Advantages:**

- Easy to compute (matrix operations)
- Suitable for linear algebra methods
- Used in:
  - Spectral clustering
  - Graph neural networks
  - PageRank

**Key idea:**

$$\text{Graph} \leftrightarrow \text{Matrix}$$

# Load Adjacency Matrix in Python

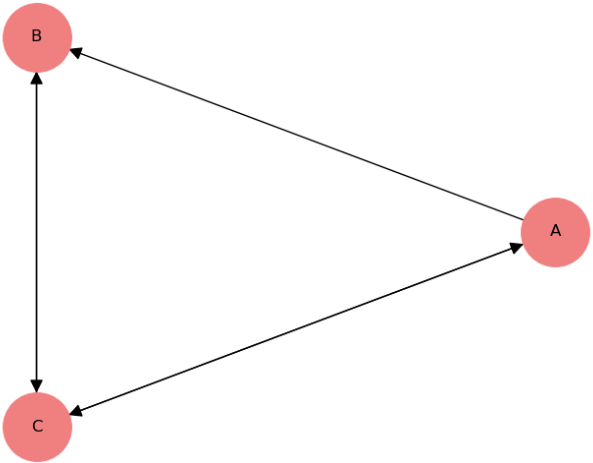
```
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
adj_matrix = np.loadtxt("simple_adj.txt", dtype=int)
G = nx.from_numpy_array(adj_matrix, create_using=nx.DiGraph())
mapping = {0: "A", 1: "B", 2: "C"}
G = nx.relabel_nodes(G, mapping)
pos = nx.circular_layout(G)
nx.draw(G, pos, with_labels=True, node_color="lightcoral", node_size=2500, arrowsize
    =20)
plt.title("Graph from Adjacency Matrix")
plt.axis("off")
plt.show()
```

# Create Graph (NetworkX)

```
[[0 1 1]
 [0 0 1]
 [1 1 0]]
```

Think about the following questions:

- Is it a directed graph and undirected graph?
- Can you draw the graph according to this adjacency matrix?




Besides network structure, what else we need to visualize in network?

# From Data to Visualization

**Key Question:**

How to reflect graph information in visualization?

**Idea: Encode information visually**

- Edge weight $\rightarrow$ **Edge thickness**
- Node degree $\rightarrow$ **Node size**
- Node importance $\rightarrow$ **Color**

**Goal**

Make invisible structure **visible**

# Edge Weight $\rightarrow$ Thickness

**Weighted Graph:**

- Edge has weight $w_{ij}$

**Visualization Idea:**

$$\text{width}_{ij} \propto w_{ij}$$

**Interpretation:**

- Thicker edge $\Rightarrow$ stronger connection
- Thinner edge $\Rightarrow$ weaker connection

**Examples:**

- Social network: interaction frequency
- Transportation: traffic volume
- Finance: transaction amount

# Degree → Node Size

**Recall:**

$$d_i = \sum_j A_{ij}$$

**Visualization Idea:**

$$\text{size}_i \propto d_i$$

**Interpretation:**

- Large node ⇒ highly connected
- Small node ⇒ less connected

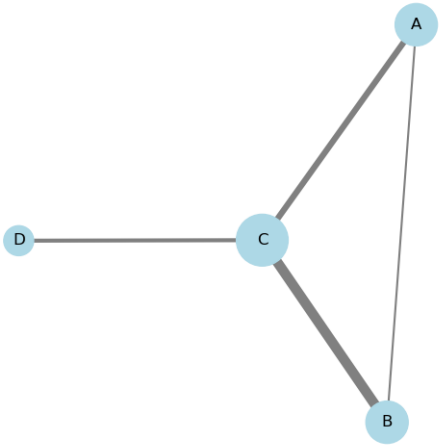
**In social networks:**

- High degree ⇒ more influence / exposure

# Visualization in Python (Complete Example)

```
import networkx as nx
import matplotlib.pyplot as plt
# 1. Create a weighted graph
G = nx.Graph()
G.add_edge("A", "B", weight=1)
G.add_edge("A", "C", weight=3)
G.add_edge("B", "C", weight=5)
G.add_edge("C", "D", weight=2)
# 2. Layout
pos = nx.spring_layout(G, seed=42)
# 3. Edge width ~ weight
weights = [G[u][v]["weight"] for u, v in G.edges()]
edge_widths = [w * 1.5 for w in weights]
# 4. Node size ~ degree
degrees = dict(G.degree())
node_sizes = [degrees[n] * 500 for n in G.nodes()]
# 5. Draw graph
plt.figure(figsize=(5,5))
nx.draw(G, pos,with_labels=True,width=edge_widths,node_size=node_sizes,node_color="
    lightblue",edge_color="grey")
plt.title("Graph Visualization")
plt.axis("off")
```

# Result From Last Slide

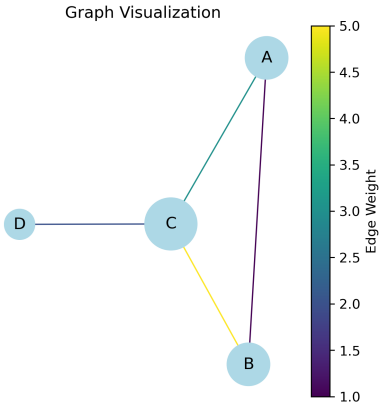



- Edge Width represents weight of edge.
- Node Size represents the degrees of nodes.

# Visualization in Python (Complete Example)

```
fig, ax = plt.subplots(figsize=(5,5))
nx.draw(G, pos,with_labels=True,
    node_size=node_sizes,
    node_color="lightblue",
    edge_color=weights,
    edge_cmap=plt.cm.viridis,
    ax=ax
)
# colorbar
sm = plt.cm.ScalarMappable(cmap=plt.cm.viridis,norm=plt.Normalize(vmin=min(weights),
     vmax=max(weights)))
sm.set_array([])
fig.colorbar(sm, ax=ax, label="Edge Weight")
plt.title("Graph Visualization")
plt.axis("off")
```

# Result From Last Slide




- Color represents weight of edge.
- Node Size represents the degrees of nodes.

# Media Network Dataset

- Source: `https://gist.github.com/abresler/23d6ead3c43f44f4d8cc56d5ef908351`

# 1. Dataset Overview: Nodes and Edges

This project uses two CSV files in the Python code:

- `nodes.csv`: stores node attributes
- `edges.csv`: stores relationships between nodes

**Dataset summary:**

- Number of nodes: 17
- Number of edges: 49
- Node categories: Newspaper, TV, Online
- Edge categories: hyperlink, mention

**Main fields:**

- Nodes: `id`, `media`, `media.type`, `type.label`, `audience.size`
- Edges: `from`, `to`, `type`, `weight`

So this dataset can be modeled as a **directed network**: each node represents a media outlet, and each edge represents a connection such as a hyperlink or a mention.

| id | media | media.type | type.label | audience.size |
|---|---|---|---|---|
| s01 | NY Times | 1 | Newspaper | 20 |
| s07 | CNN | 2 | TV | 56 |
| s12 | Yahoo News | 3 | Online | 33 |
| s14 | Reuters.com | 3 | Online | 12 |

**Example from** `edges.csv`

| from | to | type | weight |
|---|---|---|---|
| s01 | s02 | hyperlink | 22 |
| s01 | s15 | mention | 20 |
| s03 | s10 | mention | 2 |
| s12 | s14 | mention | 22 |

**Interpretation:**

- `from` $\rightarrow$ `to` shows the direction of the relationship
- `type` tells us whether it is a hyperlink or a mention
- `weight` measures the strength of the connection
- `audience.size` can be used to show the importance of a node

# 3. What Kind of Graph Do We Want to Show?

To make the graph informative, the code maps data attributes to visual elements:

- **Node color** → `type.label` Different media types are shown with different colors
- **Node size** → `audience.size` A larger audience means a larger node
- **Edge direction** → `from`, `to` This shows how information flows in the network
- **Edge weight** → `weight` It can be represented by edge width or color intensity
- **Node label** → `media` This shows the name of each media outlet

This final network graph combines structure and attributes in one visualization.

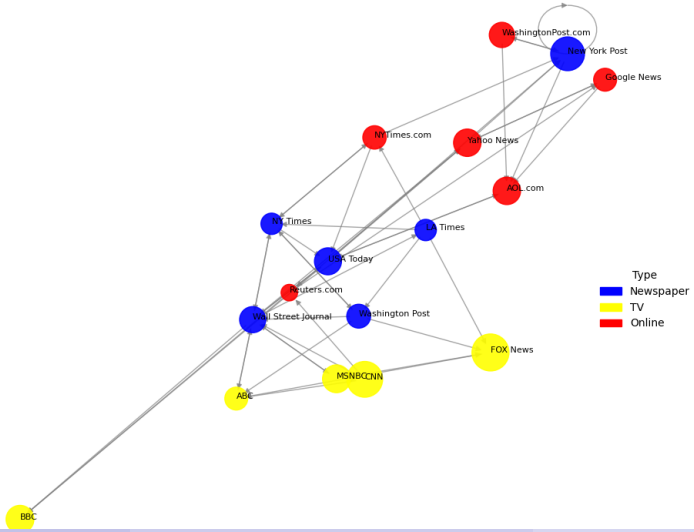
# 1. Build the Graph and Encode Node Attributes

```
import pandas as pd
nodes = pd.read_csv("nodes.csv")
edges = pd.read_csv("edges.csv")
G = nx.from_pandas_edgelist(
    edges,
    source="from",
    target="to",
    create_using=nx.DiGraph()
)
for _, row in nodes.iterrows():
    G.nodes[row["id"]].update(row.to_dict())
pos = nx.spring_layout(G, seed=42)
raw_sizes = [G.nodes[n]["audience.size"] * 20 for n in G.nodes()]

type_labels = nodes["type.label"].unique()
color_map = dict(zip(type_labels, ['blue','yellow','red']))
node_colors = [color_map[G.nodes[n]["type.label"]] for n in G.nodes()]
```

# 2. Draw the Network and Add Labels / Legends

```
from matplotlib.patches import Patch
plt.figure(figsize=(10, 8))
nx.draw_networkx_edges(G, pos, alpha=0.7, edge_color="gray", width=1)
nx.draw_networkx_nodes(G, pos,
    node_size=raw_sizes,
    node_color=node_colors,
    alpha=0.9
)
texts = []
for node, (x, y) in pos.items():
    label = G.nodes[node]["media"]
    texts.append(plt.text(x, y, label, fontsize=8, color="black"))
color_handles = [Patch(color=color_map[label], label=label) for label in type_labels
    ]
plt.legend(handles=color_handles, title="Type", loc=(0.9, 0.4), frameon=False)
plt.axis("off")
plt.tight_layout()
plt.show()
```

# 3. Final Visualization Result



# Community Detection in Network Data

**Goal:** identify groups of nodes that are more densely connected to each other than to the rest of the network.

Community detection is useful because it helps us:

- find hidden group structure in a network
- summarize a complex graph into interpretable clusters
- understand which nodes belong to the same functional group

In this example, we assume:

- edges are **undirected**
- edges are **unweighted**

So the graph only records whether two nodes are connected or not.

# What Is a Community?

A **community** is a set of nodes with relatively dense internal connections.

Intuitively:

- nodes in the same community connect frequently
- nodes in different communities connect less frequently

This idea appears in many applications:

- social networks: friend groups
- citation networks: research areas
- media networks: related outlets or information circles

Therefore, community detection is a way to discover the **meso-scale structure** of a network.

# Background: Stochastic Block Model (SBM)

A classic probabilistic model for community structure is the **Stochastic Block Model (SBM)**.

Basic idea:

- each node belongs to a latent group (block)
- the probability of an edge depends on the groups of the two nodes

If nodes $i$ and $j$ belong to communities $z_i$ and $z_j$, then

$$P(A_{ij} = 1) = B_{z_i z_j}$$

where:

- $A_{ij}$ is the adjacency matrix entry
- $z_i$ is the community label of node $i$
- $B$ is the block probability matrix

# 4. Why SBM Is Useful

SBM gives a statistical explanation for why communities appear.

For example:

- nodes inside the same block may have a high connection probability
- nodes across different blocks may have a lower connection probability

This means that a network can be viewed as generated by hidden groups.

In practice:

- SBM is a **model-based view** of communities
- many algorithms try to recover similar group structure from observed edges

In our slides, SBM is the theoretical background, while the actual implementation uses a simpler network-based community detection algorithm.

# Workflow for This Example

We use the following pipeline:

1. load nodes and edges from CSV files
2. convert the edge list into an **undirected, unweighted** graph
3. detect communities from the graph structure
4. compute node degree
5. visualize the graph:
   - color = community
   - size = degree

This design makes the final graph easy to interpret:

- color shows group membership
- size shows local importance or connectivity

# Step 1: Build an Undirected, Unweighted Graph

```
import pandas as pd
import networkx as nx

nodes = pd.read_csv("nodes.csv")
edges = pd.read_csv("edges.csv")

# Keep only connectivity and ignore direction / weight
G = nx.from_pandas_edgelist(edges,
    source="from",
    target="to",
    create_using=nx.Graph()
)
for _, row in nodes.iterrows():
    if row["id"] not in G:
        G.add_node(row["id"])
    G.nodes[row["id"]].update(row.to_dict())
```

Here, `nx.Graph()` means:

- no arrow direction
- no weight encoding

# Step 2: Detect Communities

For the actual partition, we can use **greedy modularity optimization**.

```
from networkx.algorithms.community import greedy_modularity_communities

communities = list(greedy_modularity_communities(G))

community_map = {}
for i, comm in enumerate(communities):
    for node in comm:
        community_map[node] = i
```

Why this method?

- it is simple and works directly on network structure
- it partitions nodes into communities automatically
- it is suitable for undirected and unweighted graphs

So each node receives a community label such as Community 1, 2, or 3.

# Step 3: Encode Community and Degree

After detecting communities, we map graph properties to visual variables.

```
import seaborn as sns
import numpy as np
from sklearn.preprocessing import MinMaxScaler

degrees = dict(G.degree())
raw_sizes = np.array([degrees[n] for n in G.nodes()]).reshape(-1, 1)
node_sizes = MinMaxScaler((200, 1000)).fit_transform(raw_sizes).flatten()

palette = sns.color_palette("Set2", len(communities))
node_colors = [palette[community_map[n]] for n in G.nodes()]
```

Visual mapping:

- **node color** → community label
- **node size** → degree

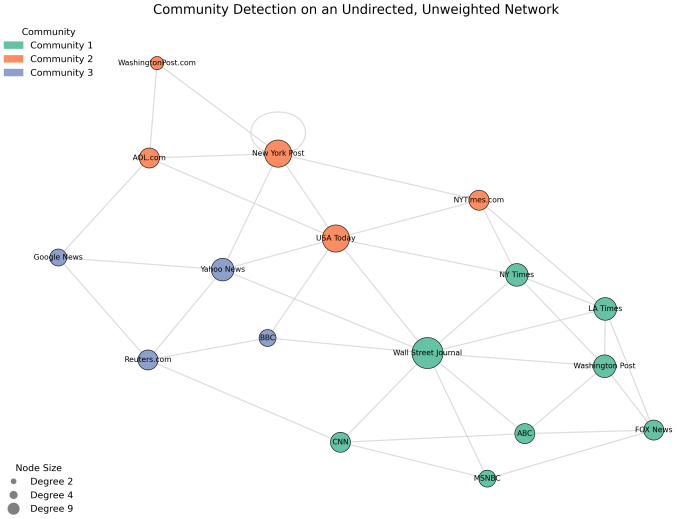
This makes clusters and central nodes visible at the same time.

# Step 4: Draw the Final Graph

```
import matplotlib.pyplot as plt

pos = nx.spring_layout(G, seed=42)

plt.figure(figsize=(10, 8))
nx.draw_networkx_edges(G, pos, edge_color="lightgray", width=1.2, alpha=0.8)
nx.draw_networkx_nodes(
    G, pos,
    node_color=node_colors,
    node_size=node_sizes,
    edgecolors="black",
    linewidths=0.5
)

labels = {n: G.nodes[n]["media"] for n in G.nodes()}
nx.draw_networkx_labels(G, pos, labels=labels, font_size=8)
plt.title("Community Detection on an Undirected, Unweighted Graph")
plt.axis("off")
plt.tight_layout()
```

The spring layout places connected nodes closer together, so community structure becomes

# Final Result



# Social Network Community Detection

- Build a social network graph from node and edge files
- Detect communities using `NetworkX`
- Visualize different communities with different colors
- Save the final network figure as an image

# Data Description

- `edges.txt`: each line contains two integers representing an edge
- `nodes.txt`: each line contains node ID, name, and gender
- Example (Node Set):

| Node ID | Name | Gender |
|---|---|---|
| 0 | LiMing | Male |
| 1 | WangFang | Female |
| 2 | ZhangWei | Male |

- Example (Edge Set):

| Node ID-1 | Node ID-2 |
|---|---|
| 0 | 1 |
| 0 | 2 |
| ... | |

- **Question**: How to visualize this dataset?

# Code: Load the Data

```
from pathlib import Path
import matplotlib
matplotlib.use('TkAgg')
import matplotlib.pyplot as plt
import networkx as nx
BASE_DIR = Path("Social_Network_Data_Anaysis")
EDGES_FILE = BASE_DIR / "edges.txt" # the location of edge file
NODES_FILE = BASE_DIR / "nodes.txt" # the location of node file
nodes = {}
with open(NODES_FILE, "r", encoding="utf-8") as f: # read the node file and add node
    attributes to nodes
    for line in f:
        line = line.strip()
        if not line:
            continue
        node_id_str, name, gender = line.split()
        node_id = int(node_id_str)
        nodes[node_id] = {"name": name, "gender": gender}
G = nx.Graph() # creat a graph and then we can add attributes and edge information
    to G.
```

# Code: Build the Network Graph

```
for node_id, attrs in nodes.items():
    G.add_node(node_id, **attrs)

with open(EDGES_FILE, "r", encoding="utf-8") as f: # read the edge file
    for line in f:
        line = line.strip()
        if not line:
            continue
        source, target = map(int, line.split())
        G.add_edge(source, target) # add each edge the graph.
```

# Graph Construction

- Use `nx.Graph()` to create an undirected graph
- Each node stores two attributes: name and gender
- Each line in the edge file adds one connection between two nodes
- The result is a complete social network structure

# Code: Community Detection

```
communities = list(nx.algorithms.community.greedy_modularity_communities(G))
communities = sorted(communities, key=lambda group: min(group))

print("Detected communities:")
for community_id, community_nodes in enumerate(communities):
    members = sorted(community_nodes)
    names = [G.nodes[node]["name"] for node in members]
    print(f"Community {community_id}:")
    print(f"  Node IDs: {members}")
    print(f"  Names: {names}")
```

# Community Detection Method

- The algorithm used is `greedy_modularity_communities()`
- It detects communities by maximizing modularity
- The goal is to find groups with dense internal links and sparse external links
- This is well suited for networks with SBM-like community structure

# Code: Assign Colors and Layout

```
community_map = {}
for community_id, community_nodes in enumerate(communities):
    for node in community_nodes:
        community_map[node] = community_id

pos = nx.spring_layout(G, seed=42, k=0.55)
cmap = plt.colormaps.get_cmap("tab10").resampled(len(communities))

node_colors = [cmap(community_map[node]) for node in G.nodes()]
labels = {node: f"{node}\n{G.nodes[node]['name']}" for node in G.nodes()}
```

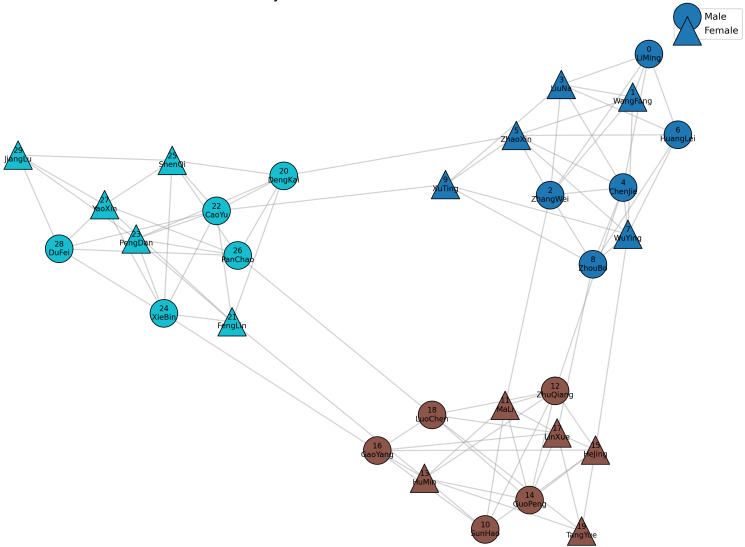
# Visualization Design

- `spring_layout` positions nodes using a force-directed layout
- Nodes in the same community share the same color
- Labels display both node ID and person name
- This helps us see clusters and bridge nodes more clearly

# Code: Draw and Save the Figure

```
plt.figure(figsize=(12, 9))
nx.draw_networkx_edges(G, pos, edge_color="#999999", alpha=0.45, width=1.2)
nx.draw_networkx_nodes(
    G,
    pos,
    node_color=node_colors,
    node_size=850,
    edgecolors="black",
    linewidths=0.8,
)
nx.draw_networkx_labels(G, pos, labels=labels, font_size=8)

plt.title("Community Detection on Social Network", fontsize=16)
plt.axis("off")
plt.tight_layout()
plt.savefig(OUTPUT_FILE, dpi=300, bbox_inches="tight")
plt.show()
```

# Result Interpretation

- The program prints the node IDs and names in each detected community
- It also saves the final graph as `community_graph.png`
- Different colors represent different communities
- If the network follows an SBM-like pattern, the clusters should appear clearly separated

Community Detection on Social Network